

프로그래밍 입문 깃허브 정리 & 카드 뽑기 게임 발표

2026864002 김동휘

깃허브 주소 :

<https://github.com/GreenRiceCake/python>

1주차: 파이썬 소개

1. 파이썬 소개

컴퓨터는 하드웨어와 소프트웨어로 이루어진 기계로 다양한 일을 할 수 있는 기계이며, 명령어의 집합인 프로그램을 통해 다양한 작업을 수행합니다. 컴퓨터는 사람의 언어를 직접 이해하지 못하기 때문에 프로그래밍 언어가 필요하며, 작성된 코드는 컴파일러나 인터프리터가 기계어로 변환해 줍니다. 파이썬은 문법이 간결하고 라이브러리가 풍부해 초보자부터 전문가까지 널리 쓰이며, 주피터 노트북·Colab 환경에서 코드와 설명을 함께 작성하고 바로 실행해 볼 수 있습니다.

예를 들자면

```
print("Hello Python")
```

`print()` 함수를 사용해 문자열이나 계산 결과를 화면에 출력하는 방법을 익혔습니다. 또한 파이썬 파일의 확장자가 `.py` 라는 점과, 파이썬은 대소문자를 구분한다는 점도 배웠습니다.

예를 들어 `print` 와 `Print` 는 서로 다르게 인식되므로 정확한 문법을 사용하는 것이 중요합니다.

Commits

History for `Python` / `0306` on `main`

All users All time

Commits on Jun 8, 2026

Update readme.md

GreenRiceCake authored 14 minutes ago

Verified

d099e65

Commits on Mar 12, 2026

Update assignment

GreenRiceCake committed on Mar 12

82d1db8

Update assignment

GreenRiceCake committed on Mar 12

89aac14

Update assignment

GreenRiceCake committed on Mar 12

045f5af

Assignment submit

GreenRiceCake committed on Mar 12

e3ce327

Assignment submit

GreenRiceCake committed on Mar 12

33e5b8a

test

GreenRiceCake committed on Mar 12

7969ee7

assignment submit

GreenRiceCake committed on Mar 12

28f6171

assignment submit

GreenRiceCake committed on Mar 12

22e4d6c

Commits on Mar 6, 2026

Create readme.md

GreenRiceCake authored on Mar 6

Verified

318cc34

End of commit history for this file

Python / 0306 /

Add file ...

GreenRiceCake Update readme.md

d099e65 · 16 minutes ago History

Name	Last commit message	Last commit date
..		
0306-2026864002.hwp	Update assignment	3 months ago
hello.py	Assignment submit	3 months ago
readme.md	Update readme.md	16 minutes ago

[illegible]

1주차: 파이썬 소개

2주차: 변수와 수식

2. 변수와 수식

변수는 값을 저장하고 사람이 식별하기 쉽게 이름을 붙인 메모리 공간입니다.
파이썬에서 변수는 직접 값을 담는다고 보다는 메모리에 저장된 객체를 가리키는 이름에 가깝습니다.

```
x = 100
y = 200
result = x + y
print(result)
```

파이썬에서는 변수에 값을 대입하면 자동으로 변수가 만들어집니다.
이 과정에서 정수, 실수, 문자열 같은 자료형을 다루는 방법을 익혔습니다.

또한 산술 연산자도 실습했습니다.

```
+ - * / // % **
```

각 연산자는 덧셈, 뺄셈, 곱셈, 나눗셈, 뮌, 나머지, 거듭제곱을 의미합니다.
입력값을 사용자에게 직접 받을 때는 `input()` 함수를 사용하며, 숫자로 계산하려면 `int()` 나 `float()` 를 이용해 형 변환을 해야 합니다.

```
num = int(input("숫자를 입력하세요: "))
```

Python / 0313 /			Add file ▾	⋮
Update readme.md			3a082f7 · 22 minutes ago	History
Name	Last commit message	Last commit date		
..				
0313-2026864002.hwp	Add files via upload	3 months ago		
circleArea.py	Add files via upload	3 months ago		
inputFunc.py	Add files via upload	3 months ago		
readme.md	Update readme.md	22 minutes ago		
sachic.py	Add files via upload	3 months ago		
triangleArea.py	Add files via upload	3 months ago		

Commits

History for Python / 0313 on `main`

All users ▾

All time ▾

⌵ Commits on Jun 8, 2026

Update readme.md

authored 23 minutes ago

Verified

3a082f7



⌵ Commits on Mar 18, 2026

Add files via upload

authored on Mar 18

Verified

0f966d3



⌵ Commits on Mar 13, 2026

Add files via upload

authored on Mar 13

Verified

00d12f6



Add files via upload

authored on Mar 13

Verified

af3e80f



Add files via upload

authored on Mar 13

Verified

0da62dc



⌵ Commits on Mar 6, 2026

Create readme.md

authored on Mar 6

Verified

af357c3



⌵ End of commit history for this file

2주차: 변수와 수식

중간점검

1. 변수를 다시 설명해보자.
변수는 메모리에 특정한 데이터를 저장하고 식별하기 쉽게 이름을 붙여둔 것이다.
2. 밑변이 10이고 높이가 10인 삼각형의 면적을 계산하는 프로그램을 작성해보자.

```
area = 0.5 * base * height
print(area)
```
3. 변수 이름을 만들 때 지켜야 하는 규칙은 무엇인가?
허용된 이름만 사용하고, 소문자와 대문자는 서로 다르게 취급된다. 그리고 변수의 이름은 영문자와 숫자, 밑줄(_)로 이루어져야 하고, 변수의 이름 중간에 공백이 들어가지면 안 된다. 단 이름 구분자라면 밑줄(_)을 사용한다.
4. 변수 이름의 첫 번째 글자로 허용되는 것은 무엇인가?
영문자와 밑줄(_)이다.
5. 파이썬에서 고유의 의미를 가지고 있는 단어들을 무엇이라고 하는가?
파이썬에서 고유한 의미를 가지고 있는 단어들은 예약어라고 하며, 예를 들어 while, for, def 처럼 이미 기능을 갖추고 있는 키워드를 말한다.
6. 파이썬에는 어떤 자료형이 있는가? 3가지만 말해보자.
정수형 int, 실수형 float, 문자열 str
7. 파이썬 변수를 정수형이라고 할 수 있을 것인가? 그 이유를 무엇인가?
가능하다. 동적 타이핑 언어이기 때문에 파이썬은 변수를 선언할 때 변수에 할당되는 값에 따라 실시간으로 자료형이 바뀐다.
8. 변수의 자료형을 알려면 어떤 함수를 사용하는가?
type() 함수를 사용한다.
9. 변수 x에 300을 저장하는 문장을 작성하여 보자.
x = 300
10. 다음 프로그램은 사각형의 면적을 계산한다. 코드에 주석을 붙여보자.

```
width = 5
height = 10
area = width * height
print(area)
```
11. 복합 대입 연산자 x += y의 의미를 설명하라.
x = x + y와 같은 의미이다. x에 y의 값을 곱한 결과를 x에 다시 저장하려는 의미이다.
12. 10%의 값은 무엇인가?
0.1이다. 10을 6으로 나눈 나머지를 구하는 수식이다.
13. 14를 6로 나눈 10/6의 값은 얼마인가?
1이다. 10을 6으로 나눈 몫을 구하는 수식이다.
14. 다음의 할당문에서 무엇이 잘못되었는가?
3 = x
할당하려는 변수가 -의 왼쪽에 와야 한다. 그래서 x = 3이 맞다.

- ```
202406-0002 김동휘
```
15. 다음의 할당문이 실행된 후에 변수 a, b, c의 값은?  
a = b = c = 100  
변수 a, b, c 모두 100이다.
  16. 문자열 "100"을 정수 100으로 변환하는 명령문을 작성하라.  

```
int("100")
```
  17. 정수 100을 문자열 "100"으로 변환하는 명령문을 작성하라.  

```
str(100)
```
  18. 다음과 같이 3개의 문자열이 3개의 변수에 저장되어 있다. 첫글자만을 따서 "BTW"라고 만들고 싶다. 어떻게 하면 되는가?  
a = "By"  
b = "The"  
c = "Way"
- ```
BTW = b[0] + c[0] + a[0]
```
19. 사용자로부터 3개의 정수를 받아서 시작연산(+, -, *, /)을 한 후에 결과를 출력하는 프로그램을 작성해보자.

```
num1 = int(input("첫 번째 정수: "))
num2 = int(input("두 번째 정수: "))
num3 = int(input("세 번째 정수: "))
result = num1 + num2 * num3
print(result)
```
 20. 자동차의 이름을 몰아보고 약해서 3개의 정수를 받아서 덧셈을 한 후에 결과를 출력하는 다음과 같은 프로그램을 작성해보자.

```
name = input("자동차의 이름을 입력하세요: ")
num1 = int(input("첫 번째 정수: "))
num2 = int(input("두 번째 정수: "))
num3 = int(input("세 번째 정수: "))
result = num1 + num2 + num3
print(result)
```

Lab: 로봇 기사 만들기

```
def robot_driver():
    name = input("로봇의 이름을 입력하세요: ")
    speed = int(input("로봇의 속도를 입력하세요: "))
    weight = int(input("로봇의 무게를 입력하세요: "))
    score = 0
    while True:
        user_input = input("로봇의 상태를 입력하세요: ")
        if user_input == "start":
            score += 10
        elif user_input == "stop":
            score -= 10
        elif user_input == "restart":
            score += 20
        elif user_input == "quit":
            break
    print("로봇의 점수는: ", score)
```

Lab: 대화하는 프로그램 만들기

```
def chat_bot():
    name = input("이름을 입력하세요: ")
    age = int(input("나이를 입력하세요: "))
    score = 0
    while True:
        user_input = input("대화 내용을 입력하세요: ")
        if user_input == "hello":
            score += 10
        elif user_input == "bye":
            score -= 10
        elif user_input == "hi":
            score += 20
        elif user_input == "quit":
            break
    print("대화의 결과는: ", score)
```

Lab: BMI 계산하기

```
def bmi_calculator():
    name = input("이름을 입력하세요: ")
    weight = int(input("체중을 입력하세요: "))
    height = int(input("신장을 입력하세요: "))
    bmi = weight / (height ** 2)
    print("BMI 계산 결과: ", bmi)
```

202406-0002 김동휘

Lab: 별까지의 거리 계산하기

```
def calculate_distance():
    name = input("별의 이름을 입력하세요: ")
    distance = float(input("별까지의 거리를 입력하세요: "))
    light_year = float(input("빛의 속도를 입력하세요: "))
    time = distance / light_year
    print("별까지의 거리는: ", time)
```

Lab: 원기둥의 부피 계산

```
def calculate_volume():
    radius = float(input("원기둥의 반지름을 입력하세요: "))
    height = float(input("원기둥의 높이를 입력하세요: "))
    volume = 3.14 * radius ** 2 * height
    print("원기둥의 부피는: ", volume)
```

Lab: 복리 계산

```
def calculate_compound_interest():
    init_money = float(input("초기 금액을 입력하세요: "))
    interest = float(input("연 이자율을 입력하세요: "))
    years = int(input("기간을 입력하세요: "))
    result = init_money * (1 + interest) ** years
    print("복리 계산 결과: ", result)
```

202406-0002 김동휘

Lab: 구의 부피 계산하기

```
def calculate_sphere_volume():
    radius = float(input("구의 반지름을 입력하세요: "))
    volume = 4/3 * 3.14 * radius ** 3
    print("구의 부피는: ", volume)
```

Lab: 자동 판매기 프로그램

```
def vending_machine():
    name = input("이름을 입력하세요: ")
    money = int(input("입금 금액을 입력하세요: "))
    change = money - 1000
    while True:
        user_input = input("구매 상품을 입력하세요: ")
        if user_input == "cola":
            change -= 1000
        elif user_input == "juice":
            change -= 1500
        elif user_input == "snack":
            change -= 2000
        elif user_input == "quit":
            break
    print("잔액은: ", change)
```

202406-0002 김동휘

이번 장에서 배운 것

- 이 변수는 값을 저장하는 상자와 같은 것으로 저장한 값은 나중에 유용하게 사용할 수 있다.
- 문자열은 큰따옴표(")나 작은 따옴표(')를 사용할 수 있다.
- 덧셈, 뺄셈, 곱셈, 나눗셈을 위하여 +, -, *, / 기호를 사용한다.
- 거듭제곱은 **이다.
- 나눗셈에서 정수로 몫을 계산하려면 // 연산자를 사용한다.
- 나눗셈에서 나머지를 계산하려면 % 연산자를 사용한다.
- 순환제곱을 높은 연산자 우선순위로 계산한다.
- "명령어"와 "값"의 차이점을 이해한다.
- 연산자의 우선 순위를 변경하려면 괄호를 사용한다.
- input() 함수를 이용하여 사용자로부터 문자열을 받을 수 있다.
- 문자열을 정수로 변환하려면 int() 함수를 사용한다.
- 문자열을 정수로 변경하려면 float() 함수를 사용한다.
- 정수나 실수를 문자열로 변경하려면 str() 함수를 사용한다.
- "for"와 "while"을 나타내는 특수 문자를 사용한다.

3주차: 조건문

3. 조건문

조건문은 특정 조건에 따라 프로그램의 실행 흐름을 바꾸는 문법입니다.
조건문이 없다면 프로그램은 여러 상황에 대응하기 힘들 것입니다.

```
score = 80

if score >= 60:
    print("합격")
else:
    print("불합격")
```

if, elif, else 를 사용하여 여러 조건을 판단하는 방법을 익혔습니다.
조건식에는 >, <, >=, <=, ==, != 같은 비교 연산자를 사용했습니다.

여러 조건을 동시에 비교할 때는 논리 연산자를 사용했습니다.

```
and # 둘다 참이면 참입니다.
or  # 둘 중 하나 혹은 이상이면 참입니다.
not # 둘다 거짓이면 참입니다.
```

파이썬에서는 중괄호 대신 들여쓰기로 코드 블록을 구분하므로, 조건문 내부의 코드는 들여쓰기를 정확히 맞춰야 합니다.

Commits

History for Python / 0320 on main

Commits on Jun 8, 2026

Update readme.md

GreenRiceCake authored 26 minutes ago

Verified929c918

Commits on Mar 22, 2026

Add files via upload

GreenRiceCake committed on Mar 22

180c4c1

Commits on Mar 20, 2026

Update fact.py

GreenRiceCake authored on Mar 20

Verified5084f92

Update sqrt.py

GreenRiceCake authored on Mar 20

Verifiedc90c2b5

Add files via upload

GreenRiceCake authored on Mar 20

Verified75b7806

Add files via upload

GreenRiceCake authored on Mar 20

Verifiedb1e41f3

Add files via upload

GreenRiceCake authored on Mar 20

Verified7709d8b

Add files via upload

GreenRiceCake authored on Mar 20

Verified1acee5b

Create readme.md

GreenRiceCake authored on Mar 20

Verified3f1fc4e

End of commit history for this file

Python / 0320 /

GreenRiceCake Update readme.md929c918 · 25 minutes agoHistory

Name	Last commit message	Last commit date
..		
0320-2026864002.hwp	Add files via upload	3 months ago
fact.py	Update fact.py	3 months ago
forEx01.py	Add files via upload	3 months ago
leapyear.py	Add files via upload	3 months ago
list.py	Add files via upload	3 months ago
matchcase.py	Add files via upload	3 months ago
range.py	Add files via upload	3 months ago
readme.md	Update readme.md	25 minutes ago
sansoo.py	Add files via upload	3 months ago
shipping_cost.py	Add files via upload	3 months ago
sqrt.py	Update sqrt.py	3 months ago

3주차: 조건문

중간점검

1. 프로그램에 사용되는 3가지의 제어구로는 어떤 것들인가?
순차구조, 선택구조, 반복구조
2. 제어문은 조건문과 반복문으로 나누어진다.
3. 다음의 조건에 해당하는 논리 연산식을 만들어 보시오. 변수는 적절하게 선언되어 있다
고 가정한다.
"나이는 25살 이상, 연봉은 3500만원 이상"
 $age \geq 25$ and $salary \geq 3500000$
4. 수식 $not\ True$ 의 값은?
 $False$

Lab: 산술 퀴즈 프로그램

```
import random
a = random.randint(1, 100)
b = random.randint(1, 100)
print(a + b)
print(a * b)
print(a - b)
print(a / b)
print(a % b)
```

Lab: 산술 퀴즈 프로그램

```
import random
a = random.randint(1, 100)
b = random.randint(1, 100)
print(a + b)
print(a * b)
print(a - b)
print(a / b)
print(a % b)
```

Lab: 산술 퀴즈 프로그램

```
import random
a = random.randint(1, 100)
b = random.randint(1, 100)
print(a + b)
print(a * b)
print(a - b)
print(a / b)
print(a % b)
```

Lab: 산술 퀴즈 프로그램

```
import random
a = random.randint(1, 100)
b = random.randint(1, 100)
print(a + b)
print(a * b)
print(a - b)
print(a / b)
print(a % b)
```

```
import random
a = random.randint(1, 100)
b = random.randint(1, 100)
print(a + b)
print(a * b)
print(a - b)
print(a / b)
print(a % b)
```

Lab: 세일 가격 계산

```
def calculate_discount(price, discount):
    if price < 100000:
        return price * (1 - discount)
    else:
        return price * (1 - discount * 0.5)
```

Lab: 물의 상태 출력하기

```
def get_state(temp):
    if temp < 0:
        return "얼음"
    elif 0 <= temp < 100:
        return "물"
    else:
        return "증기"
```

Lab: 동전 던지기 게임

```
import random
def flip_coin():
    return random.choice(['heads', 'tails'])
```

Lab: 리히터 규모

```
def calculate_damage(magnitude):
    if magnitude < 2.0:
        return "미미한 피해"
    elif 2.0 <= magnitude < 4.0:
        return "작은 피해"
    elif 4.0 <= magnitude < 6.0:
        return "중대한 피해"
    else:
        return "심각한 피해"
```

Lab: 8 매직볼

```
def magic_ball():
    return random.choice(['yes', 'no', 'maybe'])
```

Lab: 사용자 입력 검증하기

```
def validate_input(prompt):
    while True:
        user_input = input(prompt)
        if user_input.isdigit():
            return int(user_input)
        else:
            print("Invalid input. Please enter a number.")
```

Lab: 축구게임

```
def play_game():
    team1 = "Red"
    team2 = "Blue"
    score1 = 0
    score2 = 0
```

Lab: 올바른 상각형 구분

```
def is_valid_syntax(syntax):
    if syntax.startswith("if") and syntax.endswith(":"):
        return True
    else:
        return False
```

이번장에서 배운 것

- 1. 조건에 따라 실행 여부를 결정하는 조건문과 반복문의 사용 방법
- 2. `if` 문과 `elif` 문, `else` 문
- 3. `for` 문과 `while` 문
- 4. `break` 문과 `continue` 문
- 5. `try` 문과 `except` 문
- 6. `with` 문과 `as` 문
- 7. `lambda` 문과 `yield` 문

4주차: 반복문

4. 반복문

반복문은 같은 코드를 여러 번 실행할 때 사용합니다.

컴퓨터는 반복적인 작업을 매우 잘 처리하므로 반복문은 매우 중요합니다.

파이썬에서는 주로 `for` 문과 `while` 문을 사용합니다.

```
for i in range(5):  
    print("반문을 환영합니다!")
```

`for` 문은 반복 횟수가 정해져 있을 때 사용하기 좋습니다.

`range()` 함수를 사용하면 일정한 범위의 숫자를 만들 수 있습니다.

```
range(5)    # 0부터 4까지  
range(1, 6) # 1부터 5까지  
range(1, 10, 2) #1부터 9까지 2칸씩(1, 3, 5, 7, 9)
```

`while` 문은 조건이 참인 동안 계속 반복합니다.

```
count = 0  
  
while count < 5:  
    print(count)  
    count += 1
```

반복문을 통해 누적합, 반복 출력, 숫자 처리, 간단한 게임 구조 등을 연습했습니다.

또한 반복문 안에 반복문을 넣는 중첩 반복문도 다뤘습니다.

Python / 0327 /

Add file ▾

...

GreenRiceCake Update readme.md

86292ba · 28 minutes ago History

Name	Last commit message	Last commit date
..		
0327-2026864002.hwp	Add files via upload	2 months ago
byul.py	Add files via upload	3 months ago
gugudan.py	Add files via upload	3 months ago
readme.md	Update readme.md	28 minutes ago

Commits

History for Python / 0327 on `main`

All users ▾

All time ▾

⌵ Commits on Jun 8, 2026

Update readme.md

GreenRiceCake authored 29 minutes ago

Verified

86292ba



⌵ Commits on Apr 2, 2026

Add files via upload

GreenRiceCake committed on Apr 2

1710817



⌵ Commits on Mar 27, 2026

Add files via upload

GreenRiceCake authored on Mar 27

Verified

166d764



Create readme.md

GreenRiceCake authored on Mar 27

Verified

4a2684e



⌵ End of commit history for this file

4주차: 반복문

2024064-4002 입동하

중간점검

1. 프로그래밍에 반복 구조가 필요한 이유는 무엇인가?
구현되는 상황에는 반복적인 작업들이 필요하데, 만약 반복 구조가 없다면 같은 코드를 여러 번 작성하는 등 식별이 필요할거야 할 것이다.

2. 반복 구조에는 조건 반복과 횡수 반복이 있다.

3. 조건 반복과 횡수 반복을 설명해보자.
조건 반복은 특정한 조건이 만족된다면 계속 반복하는 것이고, 횡수 반복은 반복을 시작하기 전 반복의 횡수를 알 때에 특정한 횡수만큼 반복하는 것이다.

4. [1, 2, 3, 4, 5]를 저장하는 리스트 myList를 생성해보자.

```
myList = [1, 2, 3, 4, 5]
```

5. myList의 첫 번째 항목을 0으로 변경해보자.

```
myList = [1, 2, 3, 4, 5]
myList[0] = 0
```

6. myList의 마지막 항목을 0으로 변경해보자.

```
myList = [1, 2, 3, 4, 5]
myList[-1] = 0
```

7. 다음 코드의 출력을 쓰시오.

```
for i in range(1, 6, 2):
    print(i**2, i)
```

1-9
1-8
1-7
1-6
1-5

2024064-4002 입동하

8. 다음 코드의 출력을 쓰시오.

```
for i in range(1, 5, 1):
    print(i)
```

2
4
6
8

9. 다음 코드의 출력을 쓰시오.

```
for i in range(10, 0, -2):
    print("Student" + str(i))
```

Student10
Student8
Student6
Student4
Student2

10. if 문과 while 문을 비교하여 보라. 조건식이 같다면 어떻게 동작하는가?
조건식이 같은 경우에 둘은 조건식이 참이면 반복적 인의 코드를 한번만 실행하고, 거짓이면 반복적 인의 코드를 실행하지 않고, 이때 조건식이 참이여 실행 인의 코드를 실행하는 등의 방식으로 동작하지만, while문은 조건식이 참인 경우 계속 반복하고 거짓이 되는 경우 반복이 종료된다.

11. for 문과 while 문을 비교해보자. 어떤 경우에 for 문을 사용하는 것이 좋은가?
또 어떤 경우에 while 문을 사용하는 것이 좋은가?
for문은 반복 횡수나 범위가 명확할 때 사용하고, while문은 조건이 참인 동안 계속 반복하기 때문에 횡수가 명확하지 않을 때 사용한다.

12. 다음 코드의 출력을 쓰시오.

```
for i in range(3):
    for j in range(4):
        print("종아기", i, "문", j+1)
```

0 종아기 0 문 0
0 종아기 1 문 0
0 종아기 2 문 0
1 종아기 0 문 0
1 종아기 1 문 1
1 종아기 2 문 2
2 종아기 0 문 0
2 종아기 1 문 2
2 종아기 2 문 4

2024064-4002 입동하

Lab: 팩토리얼 계산하기

```
def factorial(n):
    fact = 1
    for i in range(1, n+1):
        fact = fact * i
    return fact, "팩토리얼"

팩토리얼 입력하세요: 5
결과: 120
```

도전문제

1. 팩토리얼을 구하는 재귀함수를 작성하시오.

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)
```

팩토리얼 입력하세요: 5
결과: 120

2. 팩토리얼을 구하는 재귀함수를 작성하시오.

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)
```

팩토리얼 입력하세요: 5
결과: 120

2024064-4002 입동하

Lab: 방정식의 해 구하기

```
count = 100
start = 1.4
end = 2.0
for i in range(count):
    x = start + (end-start)/count
    f = (x**2) - 4
    if abs(f-0.0) < 0.001:
        print("방정식의 해는: ", x)
```

방정식의 해는: 2.0

2024064-4002 입동하

Lab: 구구단 출력

```
for i in range(1, 10):
    for j in range(1, 10):
        print(i*j, end=" ")
    print()
```

구구단 출력: 1
1*1=1
1*2=2
1*3=3
1*4=4
1*5=5
1*6=6
1*7=7
1*8=8
1*9=9

2024064-4002 입동하

Lab: 로그인 프로그램

```
password = "12345678"
while password != "pythonista":
    password = input("로그인 비밀번호: ")
    print("로그인 실패")

로그인 비밀번호: 12345678
로그인 성공: pythonista
```

Lab: 주사위 합이 6이 되는 경우

```
for a in range(1, 7):
    for b in range(1, 7):
        if a + b == 6:
            print(f"주사위 주사위(a): {a}, 주사위 주사위(b): {b}")
```

1 번째 주사위: 1, 2 번째 주사위: 5
2 번째 주사위: 2, 2 번째 주사위: 4
3 번째 주사위: 3, 2 번째 주사위: 3
4 번째 주사위: 4, 2 번째 주사위: 2
5 번째 주사위: 5, 2 번째 주사위: 1

Lab: 모든 조합 출력하기

```
person = ["John", "Alice", "Bob", "Charlie", "David", "Eve", "Frank", "Grace", "Henry"]
for person in person:
    print(person + "의 모든 조합을 출력합니다.")
```

John의 모든 조합을 출력합니다.
Alice의 모든 조합을 출력합니다.
Bob의 모든 조합을 출력합니다.
Charlie의 모든 조합을 출력합니다.
David의 모든 조합을 출력합니다.
Eve의 모든 조합을 출력합니다.
Frank의 모든 조합을 출력합니다.
Grace의 모든 조합을 출력합니다.
Henry의 모든 조합을 출력합니다.

2024064-4002 입동하

Lab: 숫자 맞추기 게임

```
import random
target = random.randint(1, 100)
count = 0
while True:
    guess = int(input("숫자 맞추기: "))
    if guess == target:
        print("정답입니다!")
        break
    elif guess < target:
        print("숫자가 작습니다.")
    else:
        print("숫자가 큼니다.")
    count += 1
```

Lab: 숫자 맞추기 게임

```
import random
target = random.randint(1, 100)
count = 0
while True:
    guess = int(input("숫자 맞추기: "))
    if guess == target:
        print("정답입니다!")
        break
    elif guess < target:
        print("숫자가 작습니다.")
    else:
        print("숫자가 큼니다.")
    count += 1
```

2024064-4002 입동하

Lab: 초등생을 위한 산수 문제 발생기

```
def generate_math_problem():
    a = random.randint(1, 100)
    b = random.randint(1, 100)
    op = random.choice(["+", "-", "*", "/"])
    if op == "/":
        b = random.randint(1, a)
    problem = f"{a} {op} {b} = ?"
    return problem
```

문제: 100 + 20 = ?
정답: 120
문제: 100 - 20 = ?
정답: 80
문제: 100 * 2 = ?
정답: 200
문제: 100 / 2 = ?
정답: 50

2024064-4002 입동하

Lab: 소수 찾기

```
count = 1
while True:
    num = int(input("소수 찾기: "))
    is_prime = True
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        print(f"소수: {num}")
        count += 1
```

소수 찾기: 2
소수: 2
소수 찾기: 3
소수: 3
소수 찾기: 4
소수: 4
소수 찾기: 5
소수: 5
소수 찾기: 6
소수: 6
소수 찾기: 7
소수: 7
소수 찾기: 8
소수: 8
소수 찾기: 9
소수: 9
소수 찾기: 10
소수: 10

2024064-4002 입동하

Lab: 파이 계산하기

```
count = 1
divisor = 4
loop_count = 1000000
while loop_count > 0:
    pi = 4 * divisor / divisor
    divisor = divisor * 2
    loop_count = loop_count - 1
    print(f"파이: {pi}")
```

파이: 3.141592653589793

2024064-4002 입동하

Lab: 도박상의 확률

```
def calculate_probability():
    count = 0
    for i in range(1, 1000000):
        num = random.randint(1, 100)
        if num < 50:
            count += 1
    return count / 1000000
```

도박상의 확률: 0.5

2024064-4002 입동하

이번 장에서 배운 것

- 문장들을 반복 실행하기 위한 for 문과 while 문을 사용한다.
- 반복 실행되는 문장들을 실행시키기 위해 break를 사용한다.
- for 문과 while 문을 혼용하여 사용할 수 있다.
- while 문은 반복 실행되는 문장들을 실행시키기 위해 사용한다.
- 반복문의 조건식에 조건식을 포함시킬 수 있다.

5주차: 함수

5. 함수

함수는 특정 기능을 수행하는 코드를 하나로 묶은 것입니다.
반복해서 사용하는 코드를 함수로 만들면 프로그램을 더 깔끔하게 작성할 수 있습니다.
함수는 def로 정의됩니다.

```
def hello():  
    print("안녕하세요")
```

함수를 실행하려면 함수 이름을 호출합니다.

```
hello()
```

함수에는 값을 전달할 수 있으며, 이를 매개변수라고 합니다.

```
def add(x, y):  
    return x + y
```

return 을 사용하면 함수의 실행 결과를 밖으로 돌려줄 수 있습니다.
함수를 사용하면서 코드 재사용성과 프로그램 구조화의 중요성을 배웠습니다.

Python / 0410 /			Add file ▾	...
GreenRiceCake Update readme.md		b472f39 · 31 minutes ago		History
Name	Last commit message		Last commit date	
..				
0410-2026864002.hwp	Add files via upload		2 months ago	
ex.py	Add files via upload		2 months ago	
readme.md	Update readme.md		31 minutes ago	

Commits

History for Python / 0410 on main

All users ▾

All time ▾

Commits on Jun 8, 2026

Update readme.md

GreenRiceCake authored 32 minutes ago

Verified

b472f39



Commits on Apr 15, 2026

Add files via upload

GreenRiceCake committed on Apr 15

0d349e7



Commits on Apr 10, 2026

Add files via upload

GreenRiceCake authored on Apr 10

Verified

2cb8e99



Create readme.md

GreenRiceCake authored on Apr 10

Verified

368c281



End of commit history for this file

5주차: 함수

2024064-0302 김동희

중간점검

- 함수란 무엇인가?
함수란 특정 작업을 수행하는 명령어 모음에 이름을 붙인 것이다.
- 함수를 사용하는 이유는 무엇인가?
프로그램을 작성하다 보면 같은 기능을 여러 번 사용하게 되는 경우가 있는데 이때 함수를 활용하면 같은 코드를 여러 번 작성하는 등의 불편함이 없을 것이다.
- 함수에 전달되는 값은 무엇이라고 하는가?
함수에 전달되는 값은 인수라고 한다.
- 함수 안에서 전달되는 값을 받는 변수를 무엇이라고 하는가?
함수 안에서 전달되는 값을 받는 변수를 매개변수라고 한다.
- 사실적으로부터 2개의 변수를 받아서 반환하는 함수를 작성해보자.

```
def get_num():  
    a = int(input("첫번째 변수를 입력하십시오 : "))  
    b = int(input("두번째 변수를 입력하십시오 : "))  
    return a, b  
num1, num2 = get_num()  
print(num1, num2)
```
- 주어진 사각형의 면적을 계산하는 함수 `get_rect_area(w, h)`를 정의해보자.
여기서 `w`는 너비, `h`는 높이이다.

```
def get_rect_area(w, h):  
    area = w * h  
    return area
```
- `main()` 함수를 정의하고 `main()` 함수 안에서 `get_rect_area()`를 호출해보자.

```
def main():  
    width = int(input("너비를 입력하십시오 : "))  
    height = int(input("높이를 입력하십시오 : "))  
    result = get_rect_area(width, height)  
    print("사각형의 넓이 :", result)  
main()
```
- 인수와 매개 변수는 다시 한번 설명해보자.
인수는 함수를 호출할 때 함수에 전달하는 값이고, 매개변수는 함수를 정의할 때 전달된 값을 받는 변수를 말한다.

2024064-0302 김동희

- 다들요 함수란 무엇인가? 예를 들어보자.
함수의 매개변수가 있는 기본값이다. 예를들어

```
def myname(name='guest'):  
    print('hello', name)  
myname()  
myname('john')
```

이러한 코드가 있을 때 인수를 적지 않으면 `hello guest`가 출력되고 `john`이라는 인수를 적으면 `hello john`이 출력된다.
- 키워드 인수란 무엇인가? 예를 들어보자.
키워드 변수는 매개변수의 순서와 상관 없이 매개변수 이름으로 판단하여 인수를 전달하는 것이다. 예를들어

```
def id(age, name):  
    print("나이:", age, "이름:", name)  
id(name="John", age=21)
```
- 매개 변수 영역 + 기초가 있다면 무슨 의미인가?
여러개의 인수를 통틀어 받아오는 매개변수이다. 즉 인수의 개수가 정해지지 않은 인수들을 받을 수 있다.
- 값을 반환하는 키워드는 무엇인가?
return 이다.
- x와 y를 받아서 x+y, x-y 값을 반환하는 함수 `addsub(x, y)`를 정의해보자.

```
def addsub(x, y):  
    return x+y, x-y  
a, b = addsub(12, 6)  
print("x+y =", a)  
print("x-y =", b)
```
- 지역 변수와 전역 변수는 어떻게 다른가?
지역 변수는 함수 안에서 생성된 변수이고 해당 함수 내에서만 활용 가능하고, 전역 변수는 함수 밖에서 생성된 변수이고 모든 곳에서 활용 가능하다.
- 함수 안에서 전역 변수의 값을 읽을 수 있는가?
함수 안에서 전역 변수의 값을 읽는 것은 가능하다.
- 함수 안에서 전역 변수의 값을 변경하면 어떻게 되는가?
함수 안에서 전역 변수의 값을 그냥 변경하고 어떤 오류가 발생한다. 그래서 함수 안에서 전역 변수의 값을 바꾸려면 `global`을 사용하면 된다.

2024064-0302 김동희

Lab: 피자 크기 비교

```
def get_area(r):  
    return 3.14 * (r**2)  
r1 = 10  
r2 = 15  
a1 = get_area(r1)  
a2 = get_area(r2)  
print(a1, a2)
```

```
def get_area(r):  
    return 3.14 * (r**2)  
r1 = 10  
r2 = 15  
a1 = get_area(r1)  
a2 = get_area(r2)  
print(a1, a2)
```

Lab: 한영 문자열 출력 함수

```
def print_str(s):  
    for i in s:  
        print(i, end=" ")  
    print()
```

```
def print_str(s):  
    for i in s:  
        print(i, end=" ")  
    print()
```

Lab: 이분법

```
def search(arr, target):  
    left = 0  
    right = len(arr) - 1  
    while left <= right:  
        mid = (left + right) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1
```

2024064-0302 김동희

Lab: 주급 계산 프로그램

```
def get_pay(rate, hour):  
    money = 0  
    if hour > 30:  
        money = rate * 30 + 1.5 * rate * (hour - 30)  
    else:  
        money = rate * hour  
    return money  
rate = 10000  
hour = 35  
pay = get_pay(rate, hour)  
print("주급:", pay)
```

```
def get_pay(rate, hour):  
    money = 0  
    if hour > 30:  
        money = rate * 30 + 1.5 * rate * (hour - 30)  
    else:  
        money = rate * hour  
    return money  
rate = 10000  
hour = 35  
pay = get_pay(rate, hour)  
print("주급:", pay)
```

Lab: 여러 개의 값 반환

```
def get_max_min(arr):  
    max_val = arr[0]  
    min_val = arr[0]  
    for i in arr:  
        if i > max_val:  
            max_val = i  
        if i < min_val:  
            min_val = i  
    return max_val, min_val
```

```
def get_max_min(arr):  
    max_val = arr[0]  
    min_val = arr[0]  
    for i in arr:  
        if i > max_val:  
            max_val = i  
        if i < min_val:  
            min_val = i  
    return max_val, min_val
```

Lab: 구조화 프로그래밍 실습

```
def main():  
    n = int(input("정수 n을 입력하십시오 : "))  
    if n < 1:  
        print("n은 1 이상이어야 합니다.")  
        return  
    arr = [0] * n  
    for i in range(1, n):  
        arr[i] = arr[i-1] + arr[i-2]  
    print(arr)
```

이번 장에서 배운 것

- 함수가 무엇인지 학습하였다.
- 인수와 매개변수가 무엇인지를 학습하였다.
- 어떻게 함수로 인수를 전달할 수 있는지를 학습하였다.
- 여러 개의 인수를 함수로 전달하는 방법을 학습하였다.
- 함수가 값을 반환하는 방법을 학습하였다.
- 지역변수, 전역변수의 차이점에 대하여 학습하였다.
- global 키워드를 사용하여서 함수 안에서 전역변수를 사용하는 방법을 학습하였다.

6주차: 파이썬 자료구조(리스트)

6. 파이썬 자료구조(리스트)

리스트는 여러 개의 값을 순서대로 저장할 수 있는 자료구조입니다.
대괄호 []를 사용해서 만듭니다.

```
scores = [10, 20, 30, 40, 50]
```

리스트의 각 값은 인덱스를 통해 접근할 수 있습니다.
인덱스는 0부터 시작합니다.

```
print(scores[0])
```

리스트는 값을 추가, 삭제, 수정할 수 있습니다.

```
scores.append(60)
scores.remove(20)
scores[0] = 100
```

음수 인덱스를 활용하면 뒤에서부터 데이터 접근이 가능합니다.

또한 리스트의 길이, 최댓값, 최솟값, 합계를 구하는 함수도 사용했습니다.

```
append() # 리스트 뒤에 추가
insert() # 원하는 위치에 삽입
remove() # 특정 값 삭제
pop() # 마지막 또는 특정 위치 삭제
sort() # 리스트를 정렬
len() # 리스트의 길이
max() # 리스트의 최댓값
min() # 리스트의 최솟값
sum() # 합계
```

리스트의 일부를 잘라내는것도 가능한데 그것은 리스트 슬라이싱이라고 합니다.

```
a = [0, 1, 2, 3, 4]

a[1:3] # → [1, 2]
a[0:2] # → [0, 1]
a[2:] # → [2, 3, 4] (끝까지)
a[:3] # → [0, 1, 2] (처음부터)
```

리스트를 활용해 여러 개의 데이터를 한 번에 관리하는 방법을 익혔습니다.

Python / 0417 /

Upload readme.md 4d91dac · 33 minutes ago History

Name	Last commit message	Last commit date
..		
0417-2026864002.hwp	Add files via upload	2 months ago
readme.md	Upload readme.md	33 minutes ago

Commits

History for Python / 0417 on [main](#)

All users All time

Commits on Jun 8, 2026

Upload readme.md

committed 34 minutes ago

4d91dac

Commits on Apr 22, 2026

Add files via upload

committed on Apr 22

d8fb8ba

End of commit history for this file

6주차: 파이썬 자료구조(리스트)

중간점검

- 1. 함수란 무엇인가?
함수란 특정 작업을 수행하는 명령어의 모음체 이름을 붙인 것이다.
- 2. 함수를 사용하는 이유는 무엇인가?
특정 명령어들을 여러 번 사용하는 등의 일이 있을 때 계속 같은 명령어들을 입력하면 코드의 가독성도 떨어지고 번거롭기 때문에 함수로 묶어서 사용한다.

Lab: 성적 처리 프로그램

```
def avg(s):  
    sum = 0  
    for i in range(1, len(s)+1):  
        sum += s[i]  
    return sum / len(s)  
  
s = [10, 20, 30, 40, 50]  
print("평균:", avg(s))
```

Lab: 리스트에서 2번째로 큰 수 찾기

```
def find2nd(s):  
    s.sort()  
    return s[-2]
```

Lab: 콘테스트 평가

```
def rank(s):  
    s.sort()  
    return s
```

Lab: 리스트로 스택 흉내내기

```
def push(s, x):  
    s.append(x)  
  
def pop(s):  
    return s.pop()
```

Lab: 친구 관리 프로그램

```
def add(s, n, a):  
    s[n] = a  
  
def delete(s, n):  
    del s[n]
```

Lab: 리스트 슬라이싱

```
s = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
s[2:5] = [10, 20, 30, 40]
```

Lab: 리스트 변경 함수

```
def modify(s, factor):  
    for i in range(len(s)):  
        s[i] = s[i] * factor
```

Lab: 리스트 항목 사용하기

```
s = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]  
s[0:10:2]
```

Lab: 누적값 리스트 만들기

```
def cumsum(s):  
    return [sum(s[0:i+1]) for i in range(len(s))]
```

Lab: 피타고라스 삼각형

```
def pythagoras(n):  
    for i in range(1, n+1):  
        for j in range(1, i+1):  
            print(i**2 + j**2, end=" ")
```

Lab: 전치 행렬 계산

```
def transpose(s):  
    return [list(row) for row in zip(*s)]
```

Lab: Tic-Tac-Toe

```
def tic_tac_toe():  
    board = [" "]*9  
    while True:  
        print(board)
```

이번 장에서 배운 것

- 리스트는 일련의 값을 저장하는 컨테이너이다.
- 리스트의 각 요소는 인덱스라는 함수로 접근된다. 예를 들어서 (번호 요소는 mylist[i]) 된다.
- append() 메소드를 사용하여 리스트의 끝에 새로운 요소를 삽입할 수 있다.
- pop() 메소드를 사용하여 리스트의 끝의 위치에서 요소를 제거할 수 있다.
- remove() 메소드를 사용하여 리스트에서 원하는 요소를 제거할 수 있다.
- * 연산자를 사용하여 두 리스트를 연결할 수 있다.
- 종단값(인덱스)을 사용하여 특정 리스트를 만들 수 있다.
- 리스트 항목은 기존 리스트를 기반으로 새로운 리스트를 작성하는 다양한 방법이다.

7주차: 파이썬 자료구조II

7. 파이썬 자료구조II

리스트 외에도 여러 자료구조를 학습했습니다.

튜플은 리스트와 비슷하지만 한 번 만든 뒤 값을 바꿀 수 없습니다. 그래서 변경되면 안되는 데이터를 저장할 때 유용합니다. 항목이 하나인 튜플을 만들 때에는 심표를 붙여야 합니다.

```
fruits = ("apple", "banana", "grape")
```

세트는 중복을 허용하지 않고 순서가 없는 자료구조입니다.

```
numbers = {1, 2, 3}
```

딕셔너리는 키와 값의 쌍으로 데이터를 저장합니다.

```
phone = {
    "Kim": "010-1234-5678",
    "Lee": "010-1111-2222"
}
```

딕셔너리는 이름과 전화번호, 상품명과 가격처럼 서로 연결된 데이터를 저장할 때 유용합니다. 그리고 키를 이용해 값을 찾을 수도 있습니다.

```
phone["Kim"]
```

문자열도 리스트와 마찬가지로 시퀀스 자료형이므로 인덱싱, 슬라이싱, 검색 등이 가능합니다.

Python / 0508 /

GreenRiceCake

Update readme.md

080d64d · 35 minutes ago

Name	Last commit message	Last commit date
..		
0508-2026864002.hwp	Add files via upload	last month
readme.md	Update readme.md	35 minutes ago

Commits

History for Python / 0508 on [main](#)

All users All time

	Commits on Jun 8, 2026	
	Create readme.md GreenRiceCake committed 36 minutes ago 080d64d	
	Commits on May 13, 2026	
	Add files via upload GreenRiceCake committed last month 09f51cd	
	Commits on May 8, 2026	
	Create readme.md GreenRiceCake authored on May 8 Verified d368807	
	End of commit history for this file	

7주차: 파이썬 자료구조II

 중간점검

2. **리스트는 시퀀스에 속하는가?**
리스트는 시퀀스에 속한다.
3. **리스트의 특성에 어떤 것들이 있는가?**
요소를 구하기 위해 인덱스 값을 사용한다. 리스트의 요소들은 범용이 붙어 있다.
리스트는 동적으로 크기가 변할 수 있다.
리스트는 []로 괄호를 묶어 표현 가능하며, 튜플은 ()로 표현하며, 한 번 생성된 후 수정이 불가능해 불변형이다.
4. **리스트를 튜플로 바꾸려면 어떤 함수를 사용하면?**
tuple() 함수를 활용하여 변형한다.
5. **제곱과 대칭집합을 설명해보자.**
제곱은 리스트의 원소를 하나씩 꺼내서 변수에 담아 더하는 것이고, 대칭집합은 반대로 리스트에 담긴 원소를 하나씩 꺼내서 더하는 것이다.
6. **enumerate() 함수의 역할은 무엇인가?**
enumerate() 함수는 인덱스와 값을 차례로 반환해 줄 수 있다.
7. **리스트의 세트의 차이점은 무엇인가?**
리스트도 세트도 재귀성과 순서 집합으로 액세스가 가능한 반면, 세트는 특정 순서로 저장되지 않고 특정 집합으로 액세스할 수 있다.
8. **세트에 저장된 항목에 접근할 때 어떻게 사용하면?**
세트는 순서가 없기 때문에 인덱스를 사용하지 않는다.
9. **리스트와 세트 간의 교집합을 계산하는 수단을 설명해보자.**
C = A & B
10. **세트에 항목을 추가하는 함수?**
add() 함수를 사용하여 세트에 항목을 추가할 수 있다.
11. **공백 dictionary를 생성하는 명령문을 만들어보자.**
d = {}
12. **dictionary에 새로운 모든 키를 방문하는 코드를 작성해보자.**
for key in dict:
 print(key, **dict[key])
13. **dictionary의 d[key] 값을 지정하는 명령문을 만들어보자.**
d[key] = v
14. **문자열에 포함된 글자들의 코드값을 모두 어떤 함수를 호출하는가?**
ord() 함수를 사용하여 문자열에 포함된 각 글자의 코드값을 정수로 얻을 수 있다.
15. **문자열 및 값 같은 어떤 구조를 추가하는 명령문을 작성해보자.**
key = 1
val = A
d[key] = val
16. **문자열과 문자열 0의 수를 비교하려면 어떤 명령문을 사용하면?**
len()
- True

 Lab: 문자열의 공통 문자

```
s1=input("첫 번째 문자열:")
s2=input("두 번째 문자열:")
list1 = list(set(s1) & set(s2))
print("\n공통적인 글자:", end=" ")
for i in list1:
    print(i, end=" ")
```

```
첫 번째 문자열:Hello World!  
두 번째 문자열:How are you?  
공통적인 글자: H e l l o
```

 Lab: 문자열의 공통 문자

```
txt = input('구독 목록을 입력하세요: ')
words = txt.split(' ')
unique = set(words) # 집합으로 변환된 자동적으로 중복을 제거한다.
print('사용된 단어의 개수 = ', len(unique))
print(unique)
```

```
입력 텍스트: I have a dream that one day every valley
will and
mountain shall be made low

사용된 단어의 개수= 14
{'Valley', 'that', 'dream', 'and', 'shall', 'every'
'one', 'day', 'will', 'have'}
```

 Lab: 영화 사전

```
english_dict = {} # 공백, 대소문자를 구분한다
english_dict["one"] = "하나" # 대소문자에 따라 단어를 저장할 수 있다
english_dict["two"] = "둘"
english_dict["three"] = "셋"
word = input("단어를 입력하십시오: ")
print(english_dict[word])
```

단어를 입력하십시오: one
하나

 Lab: 학생 성적 처리

[illegible]

 Lab: 학생 성적 처리

```
score_dic = {
    "Kim": (99, 83, 95),
    "Lee": (88, 45, 78),
    "Choi": (25, 56, 49)
}

for name, scores in score_dic.items():
    print(name, "의 평균점수:", sum(scores)/len(scores))
```

Kim: 氏 電話番号= 92.33333333333333
 Lee: 氏 電話番号= 43.44444444444444
 Choi: 氏 電話番号= 50.0

 Lab: 단어 카운터 만들기

```
from collections import Counter
text_data = "Create the highest, grandest  
a = Counter(text_data.split())
print(a)

Counter({'you': 2, 'Create': 1, 'the':  
1, 'possible': 1, 'for': 1, 'your':  
'what': 1, 'believe': 1})
```

 Lab: 회문 검사하기

```
s = input("문자열을 입력하세요: ")
a1 = s[::1] + 문자열을 거꾸로 반전
if a == a1:
    print("회문입니다.")
else:
    print("회문이 아닙니다.")
```

문자열을 입력하시오: civic

 Lab: 머리 글자어 만들기

```
phrase = input("문자열을 입력하세요: ")
acronym = ""
for word in phrase.upper().split():
    acronym += word[0] # 단어의 첫 글자를 acronym에 추가
print(acronym)
```

문자열을 입력하시오: North Atlantic Treaty Organization

 Lab: 이메일 주소 분석

```
address=input("메일 주소를 입력하세요: ")
(id, domain) = address.split("@")
print(address)
print("아이디:" + id)
print("도메인:" + domain)
```

이메일 주소를 입력하십시오: greenrc@naver.com
greenrc@naver.com
 아이디: [greenrc](http://greenrc.com)
 비밀번호: greenrc.com

 Lab: 문자열 분석

```

sentence = input("문자열을 입력하십시오: ")
table = {"alphas":0, "digits":0, "spaces":0}
for i in sentence:
    if i.isalpha():
        table["alphas"]+=1
    if i.isdigit():
        table["digits"]+=1
    if i.isspace():
        table["spaces"]+=1
print(table)

```

```
문자열을 입력하십시오: A picture is worth a thousand words
{'alphas': 29, 'digits': 0, 'spaces': 6}
```

 Lab: 트위터 메시지 처리

```
t = "Python is very easy and powerful!"
length = len(t.split(" "))
print(length)
```

Lab: OTP 발생 프로그램

```
import random
s = "0123456789" + 대성 문자열
passlen = 4 + 패스워드 길이

p = "".join(random.sample(s, passlen ))
print(p)
```

 이번 장에서 배운 것

- 튜플은 변경 불가능한 항목들을 모아둔 곳이다.
- []를 이용하여 검색 튜플을 만들 수 있다.
-
- dict처럼 키와 값으로 이루어진다.
- dict처럼 키를 연산자로 사용하여 키와 관련된 값을 검색할 수 있다.
- dict처럼 키를 pop 메소드를 사용하여 항목을 제거한다.
-
- 세트는 고유한 값들을 저장한다.
- 세트는 set() 함수를 사용하여 생성할 수 있다.
- 세트의 add() 메소드를 사용하여 새 요소를 추가할 수 있다.

8주차: 객체와 클래스

8. 객체와 클래스

객체지향 프로그래밍에서는 데이터와 기능을 하나로 묶어 객체로 표현합니다.
클래스는 객체를 만들기 위한 설계도입니다.

```
class Car:
    def __init__(self, model, color):
        self.model = model
        self.color = color
        self.speed = 0

    def speed_up(self):
        self.speed += 10
```

`__init__()` 은 객체가 생성될 때 자동으로 실행되는 생성자입니다.
`self` 는 객체 자기 자신을 의미합니다.

클래스를 통해 자동차, 사람, 원, TV 같은 객체를 만들어 보며 속성과 메서드의 개념을 익혔습니다.

Python / 0515 / [Add file](#) [...](#)

 **GreenRiceCake**

Update readme.md

080d64d · 40 minutes ago [History](#)

Name	Last commit message	Last commit date
..		
0515-2026864002.hwp	Add files via upload	2 weeks ago
car.py	Add files via upload	3 weeks ago
circle.py	Add files via upload	3 weeks ago
cnt.py	Add files via upload	3 weeks ago
person.py	Add files via upload	3 weeks ago
readme.md	Update readme.md	40 minutes ago
tv.py	Add files via upload	3 weeks ago

Commits

History for Python / 0515 on [main](#) [All users](#) [All time](#)

Commits on Jun 8, 2026

Update readme.md

 GreenRiceCake committed 41 minutes ago

080d64d [Copy](#) [Diff](#) [View](#)

Commits on May 22, 2026

Add files via upload

 GreenRiceCake committed 2 weeks ago

3d69738 [Copy](#) [Diff](#) [View](#)

Commits on May 15, 2026

Add files via upload

 GreenRiceCake authored 3 weeks ago

[Verified](#) b1f5d1a [Copy](#) [Diff](#) [View](#)

Add files via upload

 GreenRiceCake authored 3 weeks ago

[Verified](#) a2f082d [Copy](#) [Diff](#) [View](#)

Create readme.md

 GreenRiceCake authored 3 weeks ago

[Verified](#) 327ea3c [Copy](#) [Diff](#) [View](#)

End of commit history for this file

8주차: 객체와 클래스

중간점검

- 객체 지향 프로그래밍은 객체들을 조합하여서 프로그램을 작성하는 기법이다.
- 절차 지향 프로그래밍의 가장 큰 단점은 무엇인가?
- 절차 지향 프로그래밍은 코드가 길어진다던 디버깅이 어려워지기 쉽다.
- 객체들을 다시 설명해보자.
- 객체라는 데이터와 알고리즘을 하나로 묶는 것이다.
- 클래스는 무엇인가?
- 객체에 대한 설계도를 클래스라고 한다.
- 인스턴스는 무엇인가?
- 클래스로부터 만들어지는 객체를 인스턴스라고 한다.
- 클래스와 인스턴스의 관계는 무엇인가?
- 예를 들자면 클래스는 가루집이고 인스턴스는 닭속이다.
- 클래스를 정의하는 데 사용되는 구문은 무엇인가?

```
class classOne:
    pass

8. 클래스 이름은 일반적으로 어떻게 작성하는가?
기밀 케이스나 파스칼 케이스를 사용하여 작성한다.
9. 클래스의 인스턴스를 어떻게 생성하는가?
c = classOne()
```

```
10. 인스턴스의 속성과 메소드를 어떻게 접근하는가?
속성 접근 : c.count
메소드 호출 : c.increment()
```

```
11. 메소드는 무엇인가?
메소드는 클래스 내에 정의된 함수이다.
12. self의 목적은 무엇인가?
```

self는 인스턴스 자신을 가리키며 이를 통해 메소드 안에서 인스턴스의 속성에 접근하거나 다른 메소드를 호출할 수 있다.

```
13. __init__() 메소드의 목적은 무엇인가?
```

생성자라고 하며 인스턴스가 생성될 때 자동으로 호출되어 객체의 초기 상태를 설정해준다.

```
14. 정보은닉은 왜 필요한가?
```

외부에서 객체 내부의 데이터에 직접 접근해서 값을 바꾸는 것을 막아 데이터 무결성을 유지하기 위해서이다.

```
15. 접근자와 설정자의 이름은 어떻게 정하는가?
```

접근자에는 get을 붙이거나 설정자에는 set을 붙여서 이름을 정하는 것이 보편적이다.

```
16. 클래스 안에서 어떤 변수를 private로 만들려면 어떻게 하면 되는가?
```

변수 이름 앞에 ..를 붙여서 만든다.

```
17. 객체를 할수로 전달하면 원본이 전달되는가? 아니면 복사본이 전달되는가?
```

파이썬에서 변수는 실제로 객체를 저장하지 않으며, 객체의 메모리 주소를 저장한다.

```
18. 다음과 같은 문장이 실행되면 내부적으로 어떤 일이 일어나는가?
```

```
t = Television(11, 10, True)
t > t
```

5개 새로 저장되는 것이 아니라 11, 10, True 같은 메모리 주소를 저장한다.

19. 인스턴스 변수와 클래스 변수의 차이점은 무엇인가?

인스턴스 변수는 인스턴스마다 독립적으로 가지는 변수이고, 클래스 변수는 같은 클래스 내의 모든 인스턴스가 공유하는 변수이다.

20. 파이썬에서 클래스 변수를 생성하려면 어떻게 하면 되는가?

클래스 정의 내부에서 메소드 영역 밖에 변수를 생성하면 된다.

Lab: TV 클래스 정의

```
class Television:
    def __init__(self, channel, volume, on):
        self.channel = channel
        self.volume = volume
        self.on = on

    def __str__(self):
        return f'{self.channel}, {self.volume}, {self.on}'

    def set_channel(self, channel):
        self.channel = channel

    def get_channel(self):
        return self.channel

t = Television(10, 10, True)
t.show()

t.setChannel(15)
t.show()
```

```
t 10 True
15 10 True
```

Lab: 원 클래스 정의

```
import math

class Circle:
    def __init__(self, radius = 0):
        self.radius = radius

    def getRadius(self):
        return math.pi * self.radius * self.radius

    def getArea(self):
        return 2 * math.pi * self.radius

c = Circle(10)
print(f"원의 반지름: {c.getRadius()}")
print(f"원의 넓이: {c.getArea()}")
```

```
원의 반지름 314.1592653589793
원의 넓이 628.3185307179586
```

Lab: 클래스 변수

```
class Page:
    kind = "html"

    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Page("홍길동", 20)
p2 = Page("김철수", 30)
```

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Vector2D(self.x - other.x, self.y - other.y)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __str__(self):
        return f'({self.x}, {self.y})'

v = Vector2D(0, 1)
w = Vector2D(1, 2)
u = Vector2D(1, 1)
a = u + v
print(a)
```

```
(1, 1)
```

Lab: 주사위 클래스

```
from random import randint

class Dice:
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.size = 30
        self.value = 1

    def roll_dice(self):
        return self.value

    def print_dice(self):
        print(f"주사위의 값: {self.value}")

    def roll_dice(self):
        self.value = randint(1, 6)

d = Dice(100, 100)
d.roll_dice()
d.print_dice()
```

```
주사위의 값: 3
```

이번장에서 배운 것

- 클래스는 속성과 동작으로 이루어진다. 속성은 인스턴스 변수로 표현되고 동작은 메소드로 표현된다.
- 객체를 생성하려면 생성자 메소드를 호출한다. 생성자 메소드는 __init__() 이름의 메소드이다.
- 인스턴스 변수를 정의하려면 생성자 메소드 안에서 self 변수 이름과 같이 생성한다.

```
class Car:
    def __init__(self, speed, color, model):
        self.speed = speed
        self.color = color
        self.model = model

    def drive(self):
        self.speed = 60

myCar = Car(0, "blue", "c-class")
print(f"자동차의 이름을 생성하였습니다.")
print(f"자동차의 속도는", myCar.speed)
print(f"자동차의 색상", myCar.color)
print(f"자동차의 모델명", myCar.model)
myCar.drive()
print(f"자동차의 속도는", myCar.speed)
```

```
자동차의 이름을 생성하였습니다.
자동차의 속도는 0
자동차의 색상은 blue
자동차의 모델명은 c-class
자동차의 속도는 60
```

Lab: 은행 계좌

```
class BankAccount:
    def __init__(self):
        self._balance = 0

    def withdraw(self, amount):
        self._balance -= amount
        print(f"출금 {amount} 원")
        return self._balance

    def deposit(self, amount):
        self._balance += amount
        print(f"입금 {amount} 원")
        return self._balance

a = BankAccount()
a.deposit(100)
a.withdraw(10)
```

```
입금에서 100 원 출금하였음
현재 10 원 입금하였음
```

카드 뽑기 게임



```
1  import random
2  import tkinter as tk
3  from tkinter import messagebox
4
5
6  SUITS = ("♠", "♦", "♥", "♣")
7  RANKS = ("A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K")
8  RED_SUITS = ("♦", "♥")
9
10 CARD_WIDTH = 220
11 CARD_HEIGHT = 300
12
13 deck = []
14 target_card = None
15 current_card = None
16 draw_count = 0
17
18 mode_var = None
19 title_label = None
20 card_canvas = None
21 status_label = None
22 remaining_label = None
23 count_label = None
24 target_label = None
25 history_list = None
26 draw_button = None
27
```

```
29 def make_card(suit, rank):
30     return f"{suit} {rank}"
31
32
33 def split_card(card):
34     return card.split()
35
36
37 def make_deck():
38     new_deck = []
39     for suit in SUITS:
40         for rank in RANKS:
41             new_deck.append(make_card(suit, rank))
42     random.shuffle(new_deck)
43     return new_deck
44
```

```
87 def update_screen():
88     if mode_var.get() == "target":
89         title_label.config(text="특정 카드 뽑기")
90         target_label.config(text=f"목표 카드: {target_card}")
91         start_text = "목표 카드를 찾아보세요."
92     else:
93         title_label.config(text="기본 카드 뽑기")
94         target_label.config(text="목표 카드: -")
95         start_text = "카드를 한 장 뽑아보세요."
96
97     remaining_label.config(text=f"남은 카드: {len(deck)}")
98     count_label.config(text=f"뽑은 횟수: {draw_count}")
99
100     if current_card is None:
101         status_label.config(text=start_text)
102     draw_card_image(current_card)
```

```

46 def reset_game():
47     global deck, target_card, current_card, draw_count
48
49     deck = make_deck()
50     current_card = None
51     draw_count = 0
52
53     if mode_var.get() == "target":
54         target_card = random.choice(deck)
55     else:
56         target_card = None
57
58     draw_button.config(state="normal")
59     history_list.delete(0, tk.END)
60     update_screen()

```

```

63 def draw_card():
64     global current_card, draw_count
65
66     if len(deck) == 0:
67         status_label.config(text="카드를 모두 뽑았습니다.")
68         draw_button.config(state="disabled")
69         return
70
71     current_card = deck.pop()
72     draw_count += 1
73     history_list.insert(0, f"{draw_count}. {current_card}")
74
75     if mode_var.get() == "target" and current_card == target_card:
76         status_label.config(text=f"성공! {draw_count}번 만에 {current_card} 카드를 뽑았습니다.")
77         draw_button.config(state="disabled")
78     elif len(deck) == 0:
79         status_label.config(text="마지막 카드까지 뽑았습니다.")
80         draw_button.config(state="disabled")
81     else:
82         status_label.config(text=f"{current_card} 카드를 뽑았습니다.")
83
84     update_screen()

```

```
105 def draw_card_image(card):
106     card_canvas.delete("all")
107     x0, y0 = 20, 20
108     x1, y1 = CARD_WIDTH - 20, CARD_HEIGHT - 20
109
110     card_canvas.create_rectangle(x0, y0, x1, y1, fill="white", outline="black", width=2)
111
112     if card is None:
113         card_canvas.create_text(CARD_WIDTH // 2, CARD_HEIGHT // 2, text="CARD", font=("Arial", 26, "bold"))
114         return
115
116     suit, rank = split_card(card)
117     color = "red" if suit in RED_SUITS else "black"
118     card_canvas.create_text(45, 45, text=rank, font=("Arial", 20, "bold"), fill=color)
119     card_canvas.create_text(45, 75, text=suit, font=("Arial", 22, "bold"), fill=color)
120     card_canvas.create_text(CARD_WIDTH // 2, CARD_HEIGHT // 2, text=suit, font=("Arial", 70, "bold"), fill=color)
121     card_canvas.create_text(CARD_WIDTH - 45, CARD_HEIGHT - 45, text=rank, font=("Arial", 20, "bold"), fill=color)
122     card_canvas.create_text(CARD_WIDTH - 45, CARD_HEIGHT - 75, text=suit, font=("Arial", 22, "bold"), fill=color)
```

```
125 < def show_help():
126 <     messagebox.showinfo(
127 <         "도움말",
128 <         "기본 모드: 카드를 한 장씩 뽑습니다.\n\n"
129 <         "목표 모드: 목표 카드가 나올 때까지 뽑습니다.\n\n"
130 <         "새 게임: 덱을 다시 섞고 처음부터 시작합니다.",
131 <     )
```

```

134 def main():
135     global mode_var, title_label, card_canvas, status_label
136     global remaining_label, count_label, target_label, history_list, draw_button
137
138     root = tk.Tk()
139     root.title("카드 뽑기 게임")
140     root.geometry("480x500")
141
142     mode_var = tk.StringVar(value="basic")
143
144     top_frame = tk.Frame(root, pady=8)
145     top_frame.pack()
146
147     tk.Radiobutton(top_frame, text="기본", value="basic", variable=mode_var, command=reset_game).pack(side="left")
148     tk.Radiobutton(top_frame, text="목표", value="target", variable=mode_var, command=reset_game).pack(side="left")
149     tk.Button(top_frame, text="도움말", command=show_help).pack(side="left", padx=8)
150
151     title_label = tk.Label(root, text="", font=("Malgun Gothic", 18, "bold"))
152     title_label.pack(pady=5)
153
154     middle_frame = tk.Frame(root)
155     middle_frame.pack(fill="both", expand=True, padx=15)
156
157     left_frame = tk.Frame(middle_frame)
158     left_frame.pack(side="left", expand=True)
159
160     card_canvas = tk.Canvas(left_frame, width=CARD_WIDTH, height=CARD_HEIGHT, bg="lightgray")
161     card_canvas.pack()
162
163     status_label = tk.Label(left_frame, text="", font=("Malgun Gothic", 11))
164     status_label.pack(pady=8)

```

```

166     button_frame = tk.Frame(left_frame)
167     button_frame.pack()
168     draw_button = tk.Button(button_frame, text="카드 뽑기", command=draw_card, width=10)
169     draw_button.pack(side="left", padx=4)
170     tk.Button(button_frame, text="새 게임", command=reset_game, width=10).pack(side="left", padx=4)
171     tk.Button(button_frame, text="종료", command=root.destroy, width=10).pack(side="left", padx=4)
172
173     right_frame = tk.Frame(middle_frame, padx=15)
174     right_frame.pack(side="right", fill="y")
175
176     remaining_label = tk.Label(right_frame, text="", font=("Malgun Gothic", 12))
177     count_label = tk.Label(right_frame, text="", font=("Malgun Gothic", 12))
178     target_label = tk.Label(right_frame, text="", font=("Malgun Gothic", 12))
179     remaining_label.pack(anchor="w", pady=3)
180     count_label.pack(anchor="w", pady=3)
181     target_label.pack(anchor="w", pady=3)
182
183     tk.Label(right_frame, text="뽑은 기록", font=("Malgun Gothic", 12, "bold")).pack(anchor="w", pady=(15, 3))
184     history_list = tk.Listbox(right_frame, width=18, height=13)
185     history_list.pack()
186
187     reset_game()
188     root.mainloop()

```

```

191 if __name__ == "__main__":
192     | main()

```

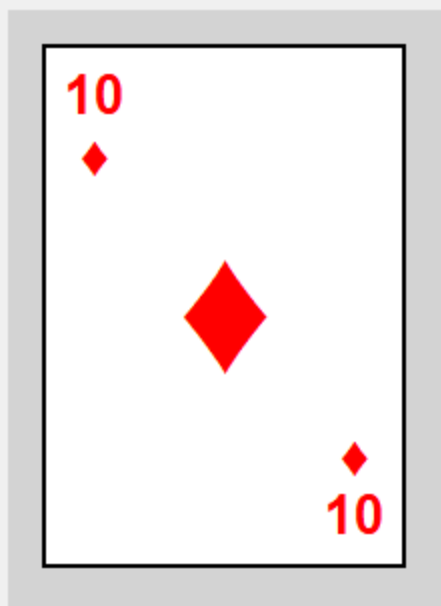


카드 뽑기 게임



☐ 기본 ☒ 목표 도움말

특정 카드 뽑기



남은 카드: 24

뽑은 횟수: 28

목표 카드: ♦ 10

뽑은 기록

28. ♦ 10
27. ♥ 2
26. ♠ 3
25. ♥ J
24. ♣ A
23. ♠ 6
22. ♦ 4
21. ♥ K
20. ♣ 10
19. ♦ Q
18. ♣ 9
17. ♥ 9
16. ♣ 4

성공! 28번 만에 ♦ 10 카드를 뽑았습니다.

카드 뽑기

새 게임

종료

느낀점

이번에 파이썬 수업에서 배운 변수, 조건문, 반복문, 함수, 리스트, 딕셔너리, 클래스 등의 내용을 바탕으로 카드 뽑기 게임 GUI 프로그램을 만들어 보면서, 단순히 문법을 외우는 것보다 직접 프로그램을 만들어 보는 것이 더 중요하다는 것을 느꼈다.

카드를 리스트로 만들고, 반복문으로 문양과 숫자를 조합하며, `random.shuffle()`로 카드를 섞는 과정은 수업에서 배운 내용을 적용해 볼 수 있어서 의미 있었다. 또한 `pop()`으로 카드를 한 장씩 뽑고 조건문으로 목표 카드와 비교하면서 리스트와 조건문이 실제 프로그램에서 어떻게 사용되는지 이해할 수 있었다.

하지만 가장 어렵게 느껴진 부분은 Tkinter를 이용해 GUI를 구성하는 과정이었다. 콘솔 프로그램과 달리 버튼, 라벨, 캔버스, 리스트박스 등을 화면에 배치해야 했고, 버튼을 눌렀을 때 함수가 실행되도록 연결하는 부분도 처음에는 복잡하게 느껴졌다. 특히 카드 그림을 Canvas에 직접 그리고, 남은 카드 수나 뽑은 횟수 같은 화면 정보를 계속 갱신하는 과정에서 GUI 프로그램은 단순히 기능만 만드는 것이 아니라 화면 구성과 사용자 조작까지 함께 생각해야 한다는 것을 알게 되었다.

이번 프로젝트를 통해 파이썬의 기본 문법들이 하나의 프로그램을 만들기 위해 함께 사용된다는 것을 알게 되었고, Tkinter를 사용하면서 사용자가 직접 조작할 수 있는 프로그램을 만드는 경험을 할 수 있었다. 앞으로는 카드 이미지 적용, 점수 시스템, 카드 맞추기 기능 등 더 다양한 기능을 추가한 프로그램도 만들어 보고 싶다.

예상 점수

GitHub : 15점

프로젝트 발표 : 15점

총합 30점

감사합니다!

